



Manual Técnico

Sistema Controle de Ronda

Versao 1.0.0

02 de Setembro de 2026

Documento Interno — CONFIDENCIAL

Equipe de Desenvolvimento BA Eletrica

suporte04@baeletrica.com.br

Sumário

- 1. Informações do Documento
 - 1.1 Histórico de Versões
 - 1.2 Público-Alvo
 - 1.3 Como Usar Este Manual
- 2. Visão Geral do Projeto
 - 2.1 O que é o Sistema
 - 2.2 Quem Usa o Sistema
 - 2.3 Stack Tecnológica
 - 2.4 Arquitetura de Alto Nível
 - 2.5 Conceitos e Terminologia
- 3. Estrutura do Projeto
 - 3.1 Mapa de Diretórios
 - 3.2 Arquivos Raiz
 - 3.3 Código-Fonte (src/)
 - 3.4 Supabase
 - 3.5 Google Apps Script
 - 3.6 GitHub Actions
 - 3.7 Onde Mexer para Cada Tipo de Alteração
- 4. Banco de Dados
 - 4.1 Diagrama Entidade-Relacionamento
 - 4.2 Tabelas
 - 4.3 Enums e Tipos
 - 4.4 Funções de Banco
 - 4.5 Triggers
 - 4.6 Storage Buckets
 - 4.7 Row-Level Security (RLS)
 - 4.8 Histórico de Migrations
- 5. API e Edge Functions
 - 5.1 Autenticação
 - 5.2 send-daily-report
 - 5.3 send-monthly-report
 - 5.4 health
 - 5.5 CORS e Headers
- 6. Frontend — Rotas e Telas
 - 6.1 Mapa de Rotas
 - 6.2 Layout Raiz

- 6.3 Rotas do Funcionário
- 6.4 Rotas do Administrador
- 7. Lógica de Negócio
 - 7.1 Ciclo de Ronda
 - 7.2 Detecção de Ciclo
 - 7.3 Filtro por Setor
 - 7.4 Sistema de Roles
 - 7.5 Timezone
- 8. Componentes
 - 8.1 Componentes Customizados
 - 8.2 Componentes shadcn/ui
- 9. Lib e Utilitários
- 10. Segurança
 - 10.1 Inventário de Chaves
 - 10.2 Variáveis de Ambiente
 - 10.3 Problemas de Segurança Identificados
 - 10.4 Recomendações de Remediação
- 11. Deploy e CI/CD
 - 11.1 Cloudflare Workers (Frontend)
 - 11.2 Supabase Edge Functions
 - 11.3 Google Apps Script
 - 11.4 GitHub Actions
 - 11.5 Sequência de Deploy
 - 11.6 Rollback
- 12. Cron e Agendamento
 - 12.1 pg_cron — Jobs Ativos
 - 12.2 Cronograma Diário
 - 12.3 Cron Mensal
 - 12.4 Diagnóstico de Falhas
- 13. Troubleshooting
 - 13.1 Tabela de Problemas Comuns
 - 13.2 Comandos de Diagnóstico
- 14. Runbooks
 - 14.1 Deploy de Edge Function
 - 14.2 Migração de Banco
 - 14.3 Rotação de Segredos
 - 14.4 Envio Manual de Relatório
- 15. Apêndices
 - 15.1 Glossário

- [15.2 Contatos e Responsáveis](#)
- [15.3 Referências](#)

1. Informações do Documento

1.1 Histórico de Versões

VERSÃO	DATA	AUTOR	ALTERAÇÕES
1.0.0	02/09/2026	Equipe Desenvolvimento	Versão inicial — documento completo

1.2 Público-Alvo

Este manual é destinado a:

- **Desenvolvedores** que precisam manter, corrigir ou evoluir o sistema
- **Administradores de sistema** que precisam fazer deploy, monitorar ou resolver incidentes
- **Gestores de TI** que precisam entender a arquitetura e capacidades do sistema
- **Novos integrantes** da equipe que precisam de contexto completo

1.3 Como Usar Este Manual

- **Capítulo 2-3:** Comece aqui para entender o projeto como um todo
- **Capítulo 4-5:** Para entender banco de dados e API
- **Capítulo 6-9:** Para mexer no frontend e lógica de negócio
- **Capítulo 10:** Para questões de segurança
- **Capítulo 11-12:** Para deploy e agendamento
- **Capítulo 13-14:** Para resolver problemas e executar rotinas
- **Capítulo 15:** Consultas rápidas

2. Visão Geral do Projeto

2.1 O que é o Sistema

O **Controle de Ronda** é um sistema web/mobile desenvolvido pela BA Elétrica para gerenciar e auditar rondas de vigilância em suas unidades (CD — Centro de Distribuição e LOJA). O sistema permite:

1. **Registro de rondas:** Funcionários batem ponto fotográfico em 10 etapas (check-in, 8 fotos intermediárias, check-out)
2. **Gestão administrativa:** Administradores visualizam registros, gerenciam usuários e setores

- 3. Relatórios automáticos:** Sistema envia PDFs profissionais com fotos por email diariamente e mensalmente
- 4. Auditoria completa:** Cada registro inclui foto com sobreposição de nome, data/hora e GPS

2.2 Quem Usa o Sistema

PERFIL	ACESSO	FUNCIONALIDADES
Admin	/admin/*	Dashboard, gestão de usuários, setores, registros, relatórios, observações
Funcionário	/app/*	Bater ponto (câmera), ver perfil, histórico de rondas
Gestor	Email	Recebe relatórios PDF diários e mensais por email

Conta admin protegida: Apenas `suporte04@baelettrica.com.br` possui role admin. Todos os outros usuários são role `user`.

2.3 Stack Tecnológica

Frontend (Produção)

CAMADA	TECNOLOGIA	VERSÃO	FUNÇÃO
Framework	TanStack Start	^1.167.50	SSR + File-based routing
UI Library	React	^19.2.0	Componentes de interface
Bundler	Vite	^7.3.1	Build e dev server
Deploy	Cloudflare Workers	-	Edge computing global
Estilo	Tailwind CSS	^4.2.1	Utility-first CSS
UI Kit	shadcn/ui (New York)	-	46 componentes prontos
Icons	Lucide React	^0.575.0	Ícones SVG
Forms	React Hook Form + Zod	^7.71.2 / ^3.24.2	Validação de formulários
Charts	Recharts	^2.15.4	Gráficos
PDF	pdf-lib	^1.17.1	Geração de PDF no client
XLSX	xlsx	^0.18.5	Planilhas Excel
Date	date-fns + date-fns-tz	^4.1.0 / ^3.2.0	Manipulação de datas com timezone
Auth	Supabase Auth	^2.106.2	Autenticação

Backend (Produção)

CAMADA	TECNOLOGIA	FUNÇÃO
Database	Supabase (PostgreSQL)	Banco de dados relacional

Auth	Supabase Auth	Autenticação e sessões
Storage	Supabase Storage	Fotos e avatares
Edge Functions	Supabase Edge Functions (Deno)	Lógica server-side
Cron	pg_cron	Agendamento de tarefas
Email (primário)	Resend API	Envio de emails transacionais
Email (fallback)	Google Apps Script (GmailApp)	Fallback de envio

Infraestrutura

COMPONENTE	TECNOLOGIA	URL
Frontend	Cloudflare Workers	<code>https://controle-ronda.suporte04.workers.dev</code>
Backend	Supabase	<code>https://rdmbayprbfqbjhfqcasp.supabase.co</code>
CI/CD	GitHub Actions	Push to main → auto-deploy
DNS	Cloudflare	<code>baelettrica.com.br</code>

2.4 Arquitetura de Alto Nível

Camada de Acesso

COMPONENTE	TECNOLOGIA	DESCRIÇÃO
Frontend	TanStack Start + React	Interface do usuário (SSR + CSR)
Deploy	Cloudflare Workers	<code>controle-ronda.suporte04.workers.dev</code>
Edge Functions	Supabase Deno	<code>send-daily-report</code> , <code>send-monthly-report</code> , <code>health</code>
Email Primário	Resend API	Envio de relatórios com anexos PDF/XLSX
Email Fallback	Google Apps Script	<code>GmailApp</code> via <code>suporte.baelettrica@gmail.com</code>

Camada de Dados (Supabase)

SERVIÇO	FUNÇÃO	SEGURANÇA
PostgreSQL	Banco de dados principal	RLS (Row-Level Security) ativo
Storage	Fotos de rondas e avatares	Bucket <code>fotos_ponto</code> (privado), <code>avatars</code> (público)
pg_cron	Agendamento de relatórios	CD 07:00, LOJA 07:05, Mensal dia 1 08:00
Auth	Autenticação de usuários	JWT + Roles (funcionario, admin, gestor)

Fluxo Geral



Fluxo de Dados — Relatório Diário

ETAPA	OPERAÇÃO	DETALHES
1	pg_cron dispara	07:00 (CD) ou 07:05 (LOJA) horário de Manaus
2	net.http_post	Chama Edge Function <code>send-daily-report</code>
3	Query registros_ponto	Seleciona registros das últimas 24h
4	Join profiles + setores	Enriquece dados com nome e setor
5	Filtrar por setor	CD (<code>73a5d2ca</code>) ou LOJA (<code>ad1b42c1</code>)
6	Buscar destinatários	Admins com role GESTOR no setor correspondente
7	Download fotos	Limitado a 40 fotos (URLs assinadas)
8	Gerar PDF	<code>pdf-lib</code> — capa, tabela, badges, fotos
9	Gerar XLSX	<code>xlsx</code> — planilha para análise
10	[PRIMÁRIO] Resend API	Envia email com anexos
11	[SE FALHAR] GAS	Google Apps Script → GmailApp
12	Retorno JSON	Status do envio

2.5 Conceitos e Terminologia

TERMO	DEFINIÇÃO
Ronda	sequência completa de 10 registros fotográficos (check-in → 8 meios → check-out)
Ciclo	Uma ronda completa. O sistema suporta múltiplos ciclos por dia
Setor	Unidade de trabalho: CD (Centro de Distribuição) ou LOJA
Passo	Cada etapa individual da ronda (check_in, meio1-8, check_out_2)
Bater Ponto	Ação de registrar um passo da ronda com foto
Gestor	Usuário administrador que recebe relatórios por email

Edge Function	Função server-side rodando no Supabase (Deno runtime)
pg_cron	Agendamento de tarefas via PostgreSQL (extensão cron)
Resend	Serviço de envio de emails transacionais (API REST)
GAS	Google Apps Script — fallback para envio de emails via GmailApp

3. Estrutura do Projeto

3.1 Mapa de Diretórios

```

controle-ronda/
├── .github/
│   └── workflows/
│       ├── deploy.yml           # Deploy automático para Cloudflare Workers
│       └── supabase-reports.yml # Keep-alive do Supabase
├── docs/
│   └── MANUAL_TECNICO.md        # Este documento
├── google-apps-script/
│   ├── Code.gs                 # Script GAS (email fallback)
│   └── appsscript.json          # Manifesto GAS
├── public/
│   └── logo.png                 # Logo BA Elétrica
├── src/
│   ├── components/
│   │   ├── AdminSidebar.tsx    # Sidebar admin
│   │   ├── CameraCapture.tsx  # Captura de câmera
│   │   ├── EmployeeBottomNav.tsx # Nav inferior funcionário
│   │   ├── ThemeToggle.tsx    # Toggle dark/light
│   │   └── ui/                 # 46 componentes shadcn/ui
│   ├── hooks/
│   │   └── use-mobile.tsx      # Detecção mobile
│   ├── integrations/
│   │   └── supabase/
│   │       ├── auth-attacher.ts # Auth attacher
│   │       ├── auth-middleware.ts # Auth middleware
│   │       ├── client.ts        # Cliente Supabase (client-side)
│   │       ├── client.server.ts # Cliente Supabase (server-side, service_role)
│   │       └── types.ts         # Tipos gerados do banco
│   ├── lib/
│   │   ├── api/
│   │   │   └── example.functions.ts # Exemplo de server function
│   │   ├── access.functions.ts     # Sincronização de acesso
│   │   ├── admin-users.functions.ts # CRUD de usuários admin
│   │   ├── auth.tsx               # Contexto de autenticação
│   │   ├── config.ts              # Config (SUPPORT_EMAIL)
│   │   ├── config.server.ts       # Config server-side
│   │   ├── date-filters.ts        # Filtros de data
│   │   ├── error-capture.ts       # Captura de erros
│   │   ├── error-page.ts          # Página de erro HTML
│   │   ├── lovable-error-reporting.ts # Reporting de erros
│   │   ├── photoOverlay.ts        # Sobreposição em Canvas
│   │   ├── report.functions.ts     # Chamada ao Edge Function
│   │   ├── storage.ts             # URLs assinadas
│   │   ├── theme.tsx              # Contexto de tema
│   │   ├── timezone.ts            # Utilitários Manaus timezone
│   │   └── utils.ts               # cn() utility
│   ├── routes/
│   │   ├── __root.tsx            # Layout raiz
│   │   ├── index.tsx             # Landing page
│   │   ├── login.tsx             # Login
│   │   ├── app.tsx               # Layout funcionário
│   │   ├── app.index.tsx         # Dashboard funcionário
│   │   └── app.perfil.tsx        # Perfil

```

```

├── app.historico.tsx      # Histórico
├── admin.tsx             # Layout admin
├── admin.index.tsx       # Dashboard admin
├── admin.usuarios.tsx    # Gestão de usuários
├── admin.setores.tsx     # Gestão de setores
├── admin.registros.tsx   # Registros de ronda
├── admin.observacoes.tsx # Observações
├── admin.relatorio-ronda.tsx # Relatório de ronda
├── admin.ronda-detalhe.$id.$inicio.tsx # Detalhe da ronda
├── routeTree.gen.ts      # Route tree auto-gerado
├── server.ts             # Server entry
├── start.ts              # App start
├── router.tsx            # Router config
├── supabase/
│   ├── config.toml       # Config Supabase
│   ├── functions/
│   │   ├── health/index.ts # Health check
│   │   ├── send-daily-report/index.ts # Relatório diário (1208 linhas)
│   │   └── send-monthly-report/index.ts # Relatório mensal (929 linhas)
│   └── migrations/       # 13 migrations SQL
├── .env.example          # Template de variáveis
├── .gitignore
├── components.json        # Config shadcn/ui
├── docker-compose.yml     # Docker (legado)
├── eslint.config.js
├── nginx.conf             # Nginx (legado)
├── package.json
├── package-lock.json
├── tsconfig.json
├── vite.config.ts
└── wrangler.toml         # Config Cloudflare Workers

```

3.2 Arquivos Raiz

ARQUIVO	FUNÇÃO	DEPLOYADO?
<code>wrangler.toml</code>	Config Cloudflare Workers	Sim (Workers)
<code>package.json</code>	Dependências e scripts NPM	Sim (build)
<code>tsconfig.json</code>	Config TypeScript	Sim (build)
<code>vite.config.ts</code>	Config Vite	Sim (build)
<code>eslint.config.js</code>	Linter	Não (dev only)
<code>.prettierrc</code>	Formatter	Não (dev only)
<code>components.json</code>	Config shadcn/ui	Sim (build)
<code>docker-compose.yml</code>	Stack legado	Não (legado)
<code>nginx.conf</code>	Nginx legado	Não (legado)
<code>.env.example</code>	Template env vars	Não (referência)
<code>.gitignore</code>	Ignorados pelo git	N/A

3.3 Código-Fonte (src/)

Rotas

ARQUIVO	ROTA	DESCRIÇÃO
<code>__root.tsx</code>	<code>/</code>	Layout raiz (AuthProvider, ThemeProvider, Toaster)
<code>index.tsx</code>	<code>/</code>	Landing page / redirect
<code>login.tsx</code>	<code>/login</code>	Tela de login (Supabase Auth)
<code>app.tsx</code>	<code>/app</code>	Layout funcionário (bottom nav)
<code>app.index.tsx</code>	<code>/app</code>	Dashboard — "Bater Ponto"
<code>app.perfil.tsx</code>	<code>/app/perfil</code>	Perfil do funcionário
<code>app.historico.tsx</code>	<code>/app/historico</code>	Histórico de rondas
<code>admin.tsx</code>	<code>/admin</code>	Layout admin (sidebar)
<code>admin.index.tsx</code>	<code>/admin</code>	Dashboard admin
<code>admin.usuarios.tsx</code>	<code>/admin/usuarios</code>	Gestão de usuários (993 linhas)
<code>admin.setores.tsx</code>	<code>/admin/setores</code>	Gestão de setores
<code>admin.registros.tsx</code>	<code>/admin/registros</code>	Registros de ronda (851 linhas)
<code>admin.observacoes.tsx</code>	<code>/admin/observacoes</code>	Observações (256 linhas)
<code>admin.relatorio-ronda.tsx</code>	<code>/admin/relatorio-ronda</code>	Relatório de ronda (499 linhas)
<code>admin.ronda-detalhe.\$id.\$inicio.tsx</code>	<code>/admin/ronda-detalhe/:id/:inicio</code>	Detalhe da ronda

Lib

ARQUIVO	FUNÇÃO PRINCIPAL	EXPORTAÇÕES
<code>auth.tsx</code>	Autenticação	<code>AuthProvider</code> , <code>useAuth()</code>
<code>config.ts</code>	Configuração	<code>SUPPORT_EMAIL</code>
<code>config.server.ts</code>	Config server	Variáveis de ambiente server-side
<code>timezone.ts</code>	Timezone Manaus	<code>toManausISO()</code> , <code>formatManaus()</code> , <code>CICLO_RONDA</code> , <code>proximaAcao()</code> , <code>TipoAcao</code>
<code>storage.ts</code>	Storage	<code>getSignedPhotoUrl()</code>
<code>photoOverlay.ts</code>	Canvas	<code>overlayPhoto()</code>
<code>date-filters.ts</code>	Filtros data	Predefined date ranges
<code>report.functions.ts</code>	Reports	<code>sendReport()</code>

<code>admin-users.functions.ts</code>	CRUD Users	<code>createUser()</code> , <code>updateUser()</code> , <code>deleteUser()</code>
<code>access.functions.ts</code>	Access	<code>syncUserAccess()</code>
<code>utils.ts</code>	Utils	<code>cn()</code>
<code>theme.tsx</code>	Tema	<code>ThemeProvider</code>

3.4 Supabase

CAMINHO	FUNÇÃO
<code>config.toml</code>	Configuração do projeto Supabase
<code>functions/health/index.ts</code>	Health check (keep-alive)
<code>functions/send-daily-report/index.ts</code>	Relatório diário (1208 linhas)
<code>functions/send-monthly-report/index.ts</code>	Relatório mensal (929 linhas)
<code>migrations/</code>	13 migrations SQL (histórico completo)

3.5 Google Apps Script

ARQUIVO	FUNÇÃO
<code>Code.gs</code>	Script principal — envio de emails via GmailApp com PDF
<code>appsscript.json</code>	Manifesto — scopes, runtime V8, timezone America/Manaus

Deploy GAS: Editor GAS → Executar manualmente → Criar implantação → Web App **Conta GAS:**

`suporte.baeletrica@gmail.com` (pessoal Gmail, NÃO Workspace) **URL de implantação:**

`https://script.google.com/macros/s/AKfycbw- ... /exec`

3.6 GitHub Actions

WORKFLOW	TRIGGER	FUNÇÃO
<code>deploy.yml</code>	Push to main	Build + Deploy para Cloudflare Workers
<code>supabase-reports.yml</code>	Cron diário 06:00 UTC	Keep-alive do endpoint health

3.7 Onde Mexer para Cada Tipo de Alteração

TIPO DE ALTERAÇÃO	ARQUIVO(S) A MODIFICAR
Adicionar nova rota	<code>src/routes/nova-rota.tsx</code>
Mudar aparência	<code>src/components/ui/*</code> ou <code>tailwind</code> classes

Mudar lógica de ronda	<code>src/lib/timezone.ts</code>
Mudar ciclo (adicionar passo)	<code>src/lib/timezone.ts</code> → <code>CICLO_RONDA</code>
Mudar filtros de data	<code>src/lib/date-filters.ts</code>
Mudar envio de email	<code>supabase/functions/send-daily-report/index.ts</code>
Mudar PDF do relatório	<code>supabase/functions/send-daily-report/index.ts</code> → <code>buildPdf()</code>
Mudar layout email	<code>supabase/functions/send-daily-report/index.ts</code> → <code>buildEmailHtml()</code>
Mudar destinatários	<code>supabase/functions/send-daily-report/index.ts</code> → <code>fetchRecipientEmails()</code>
Mudar cron	<code>supabase/migrations/</code> (SQL do pg_cron)
Mudar autenticação	<code>src/lib/auth.tsx</code> , <code>src/integrations/supabase/client*.ts</code>
Mudar RLS policies	<code>supabase/migrations/</code> (SQL das policies)
Mudar schema do banco	<code>supabase/migrations/</code> (novo arquivo SQL)
Mudar deploy frontend	<code>wrangler.toml</code> , <code>.github/workflows/deploy.yml</code>
Mudar fallback email GAS	<code>google-apps-script/Code.gs</code>
Mudar segredos Resend	Supabase Dashboard → Edge Functions → Secrets
Mudar segredos Cloudflare	<code>wrangler secret put CLOUDFLARE_API_TOKEN</code>
Adicionar componente UI	<code>npx shadcn@latest add componente</code>
Mudar config shadcn	<code>components.json</code>
Mudar dependências	<code>package.json</code>

4. Banco de Dados

4.1 Diagrama Entidade-Relacionamento

```

erDiagram
    auth_users ||--o{ profiles : "id = user_id"
    auth_users ||--o{ user_roles : "id = user_id"
    auth_users ||--o{ registros_ponto : "id = user_id"
    profiles }o--|| setores : "setor_id = id"

    profiles {
        uuid id PK "FK → auth.users"
        text nome "Nome completo"
        text email "Email corporativo"
        uuid setor_id FK "FK → setores"
        text foto_url "URL do avatar"
        timestampz created_at
        timestampz updated_at
    }

    user_roles {
        uuid id PK
        uuid user_id FK "FK → auth.users"
        app_role role "admin | user"
    }

    registros_ponto {
        uuid id PK
        uuid user_id FK "FK → auth.users"
        tipo_acao_ponto tipo_acao "check_in, meio1-8, check_out_2"
        timestampz horario_acao "Horário da ação"
        timestampz horario_foto "Horário da foto"
        text foto_url "Caminho da foto no Storage"
        text observacoes "Observação opcional"
        timestampz created_at
    }

    setores {
        uuid id PK
        text nome "Nome único do setor"
        timestampz created_at
    }

```

4.2 Tabelas

4.2.1 profiles

Tabela de perfis dos usuários. Criada automaticamente quando um usuário se cadastra.

COLUNA	TIPO	CONSTRAINTS	DESCRIÇÃO
--------	------	-------------	-----------

<code>id</code>	UUID	PK, FK → auth.users, ON DELETE CASCADE	ID do usuário
<code>nome</code>	TEXT	NOT NULL	Nome completo
<code>email</code>	TEXT	NOT NULL	Email corporativo
<code>setor_id</code>	UUID	FK → setores, ON DELETE SET NULL	Setor do usuário
<code>foto_url</code>	TEXT	nullable	Caminho do avatar no Storage
<code>created_at</code>	TIMESTAMPTZ	DEFAULT now()	Data de criação
<code>updated_at</code>	TIMESTAMPTZ	DEFAULT now()	Última atualização

4.2.2 `user_roles`

Roles dos usuários. Um usuário pode ter múltiplas roles.

COLUNA	TIPO	CONSTRAINTS	DESCRIÇÃO
<code>id</code>	UUID	PK, DEFAULT gen_random_uuid()	ID do registro
<code>user_id</code>	UUID	FK → auth.users, ON DELETE CASCADE	ID do usuário
<code>role</code>	app_role	NOT NULL	'admin' ou 'user'

Constraint: UNIQUE(user_id, role) — um usuário não pode ter a mesma role duas vezes.

4.2.3 `registros_ponto`

Registros de pontos das rondas. Cada linha = uma foto/etapa da ronda.

COLUNA	TIPO	CONSTRAINTS	DESCRIÇÃO
<code>id</code>	UUID	PK, DEFAULT gen_random_uuid()	ID do registro
<code>user_id</code>	UUID	FK → auth.users, ON DELETE CASCADE	ID do vigilante
<code>tipo_acao</code>	tipo_acao_ponto	NOT NULL	Etapa da ronda
<code>horario_acao</code>	TIMESTAMPTZ	NOT NULL	Horário que a ação foi registrada
<code>horario_foto</code>	TIMESTAMPTZ	DEFAULT now()	Horário que a foto foi tirada
<code>foto_url</code>	TEXT	NOT NULL	Caminho da foto no Storage
<code>observacoes</code>	TEXT	nullable	Observação opcional
<code>created_at</code>	TIMESTAMPTZ	DEFAULT now()	Data de criação

4.2.4 `setores`

Setores de trabalho (CD, LOJA, GESTOR, etc.).

COLUNA	TIPO	CONSTRAINTS	DESCRIÇÃO
id	UUID	PK, DEFAULT gen_random_uuid()	ID do setor
nome	TEXT	UNIQUE, NOT NULL	Nome do setor
created_at	TIMESTAMPTZ	DEFAULT now()	Data de criação

Setores existentes:

ID	NOME
e49fba28- ...	DEPARTAMENTO TI
ed4cc5bb- ...	GESTOR
73a5d2ca- ...	CD - GUARDAS
ad1b42c1- ...	LOJA - GUARDAS
bef20765- ...	GESTOR - LOJA
bc749fe5- ...	GESTOR - CD

4.3 Enums e Tipos

app_role

```
CREATE TYPE app_role AS ENUM ('admin', 'user');
```

tipo_acao_ponto

```
CREATE TYPE tipo_acao_ponto AS ENUM (
  'check_in',    -- Início da ronda
  'meio1',      -- Foto intermediária 1
  'meio2',      -- Foto intermediária 2
  'meio3',      -- Foto intermediária 3
  'meio4',      -- Foto intermediária 4
  'meio5',      -- Foto intermediária 5
  'meio6',      -- Foto intermediária 6
  'meio7',      -- Foto intermediária 7
  'meio8',      -- Foto intermediária 8
  'check_out_1',-- Check-out intermediário (legado)
  'check_out_2' -- Fim da ronda
);
```


4.4 Funções de Banco

`has_role(role app_role) RETURNS boolean`

Verifica se o usuário atual tem uma role específica. Usa `SECURITY DEFINER` para evitar recursão de RLS.

```
CREATE OR REPLACE FUNCTION public.has_role(role app_role)
RETURNS boolean
LANGUAGE sql
SECURITY DEFINER
STABLE
AS $$
    SELECT EXISTS (
        SELECT 1 FROM public.user_roles
        WHERE user_id = auth.uid() AND role = $1
    );
$$;
```

Uso em RLS policies: `SELECT public.has_role('admin')`

`handle_new_user() RETURNS trigger`

Trigger executado quando um novo usuário é criado no `auth.users`. Cria o profile e asigna role `user`.

```
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
AS $$
BEGIN
    INSERT INTO public.profiles (id, nome, email)
    VALUES (NEW.id, COALESCE(NEW.raw_user_meta_data->'nome', 'Usuário'), NEW.email);
    INSERT INTO public.user_roles (user_id, role)
    VALUES (NEW.id, 'user');
    RETURN NEW;
END;
$$;
```

`update_updated_at() RETURNS trigger`

Trigger que atualiza automaticamente a coluna `updated_at` na tabela `profiles`.

```
CREATE OR REPLACE FUNCTION public.update_updated_at()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    NEW.updated_at = now();
    RETURN NEW;
END;
$$;
```

4.5 Triggers

TRIGGER	TABELA	EVENTO	FUNÇÃO
<code>on_auth_user_created</code>	<code>auth.users</code>	AFTER INSERT	<code>handle_new_user()</code>
<code>update_profiles_updated_at</code>	<code>profiles</code>	BEFORE UPDATE	<code>update_updated_at()</code>

4.6 Storage Buckets

BUCKET	ACESSO	USO
<code>fotos_ponto</code>	Privado (authenticated read, own folder)	Fotos das rondas
<code>avatars</code>	Público	Fotos de perfil
<code>assinaturas</code>	Privado (legado)	Assinaturas
<code>comprovantes_entrega</code>	Privado (legado)	Comprovantes

Estrutura de Caminhos

```
fotos_ponto/
├─ {user_id}/
│   ├─ {timestamp}_check_in.jpg
│   ├─ {timestamp}_meio1.jpg
│   └─ ...
avatars/
├─ {user_id}/
│   └─ avatar.jpg
```

4.7 Row-Level Security (RLS)

RLS está habilitado em **todas** as tabelas. Políticas principais:

`profiles`

POLÍTICA	OPERAÇÃO	CONDIÇÃO
----------	----------	----------

<code>profiles_select_own</code>	SELECT	<code>auth.uid() = id</code>
<code>profiles_select_admin</code>	SELECT	<code>has_role('admin')</code>
<code>profiles_insert_own</code>	INSERT	<code>auth.uid() = id</code>
<code>profiles_update_own</code>	UPDATE	<code>auth.uid() = id</code>
<code>profiles_update_admin</code>	UPDATE	<code>has_role('admin')</code>

registros_ponto

POLÍTICA	OPERAÇÃO	CONDIÇÃO
<code>registros_insert_own</code>	INSERT	<code>auth.uid() = user_id</code>
<code>registros_select_own</code>	SELECT	<code>auth.uid() = user_id</code>
<code>registros_select_admin</code>	SELECT	<code>has_role('admin')</code>

user_roles

POLÍTICA	OPERAÇÃO	CONDIÇÃO
<code>user_roles_select_own</code>	SELECT	<code>auth.uid() = user_id</code>
<code>user_roles_select_admin</code>	SELECT	<code>has_role('admin')</code>
<code>user_roles_insert_admin</code>	INSERT	<code>has_role('admin')</code>

setores

POLÍTICA	OPERAÇÃO	CONDIÇÃO
<code>setores_select_auth</code>	SELECT	<code>auth.role() = 'authenticated'</code>
<code>setores_insert_admin</code>	INSERT	<code>has_role('admin')</code>

4.8 Histórico de Migrations

#	ARQUIVO	DATA	PROPÓSITO
1	<code>20260601145649_...</code>	01/06/2026	Schema inicial: profiles, user_roles, registros_ponto, setores, RLS, policies, trigger
2	<code>20260601145710_...</code>	01/06/2026	Restringir acesso a storage (revoke PUBLIC, fotos read only authenticated)
3	<code>20260605003344_...</code>	05/06/2026	Restringir SELECT storage para own folder; bloquear INSERT em user_roles para não-admins
4	<code>20260609130218_...</code>	09/06/2026	Comentário: relatórios diários via pg_cron
5	<code>20260610120000_...</code>	10/06/2026	Fix abrangente de segurança: recriar enums, tabelas, RLS, policies, storage, revoke EXECUTE

6	20260612000000_...	12/06/2026	Cleanup: manter apenas usuário suporte04 (admin)
7	20260612010000_...	12/06/2026	Adicionar foto_url ao profiles + bucket avatars
8	20260715000000_...	15/07/2026	Estender ciclo: meio1-meio8 + coluna observacoes
9	20260715120000_...	15/07/2026	Comentário: relatórios mensais via pg_cron
10	20260728000000_...	28/07/2026	Criar pg_cron jobs: diário (11:00 UTC) + mensal (12:00 UTC dia 1)
11	20260729000000_...	29/07/2026	Corrigir pg_cron: jobs separados CD (11:00) e LOJA (11:02)
12	20260818000000_...	18/08/2026	Fix todos os Supabase Advisors warnings
13	20260818120000_...	18/08/2026	Fix restante: has_role EXECUTE, dropar policies amplas de storage

5. API e Edge Functions

5.1 Autenticação

As Edge Functions do Supabase usam dois mecanismos:

1. **Service Role Key:** Acessa o banco diretamente sem RLS (via `SUPABASE_SERVICE_ROLE_KEY`)
2. **Cron Auth:** O pg_cron envia requests com o JWT anon key no header `Authorization`

Para chamadas manuais, use o header:

```
Authorization: Bearer {SUPABASE_SERVICE_ROLE_KEY}
```

5.2 send-daily-report

URL: `POST https://rdbbayprbfqbjhfcasp.supabase.co/functions/v1/send-daily-report`

Parâmetros (Body JSON)

PARÂMETRO	TIPO	OBRIGATÓRIO	DESCRIÇÃO
modo	string	Não	Ignorado (hardcoded "diario")
setor	string	Não	"CD" ou "LOJA". Se omitido, envia ambos
periodo	string	Não	Formato "YYYY-MM-DD/YYYY-MM-DD" ou "hoje_ontem". Se omitido, usa últimas 24h
override_email	string	Não	Email para teste (ignorado em produção)

Headers

```
Content-Type: application/json
Authorization: Bearer {SUPABASE_ANON_KEY ou SERVICE_ROLE_KEY}
```

Resposta (JSON)

```
{
  "ok": true,
  "modo": "diario",
  "setor": "CD",
  "periodo": "25/08/2026 07:00 a 26/08/2026 06:59 (America/Manaus)",
  "count": 60,
  "recipients": ["email1@baeetrica.com.br", "..."],
  "id": "uuid-do-email"
}
```

Fluxo de Execução (detalhado)

1. Parse body → setorParam, periodoParam
2. rangeFor(modo, periodoParam) → fromUtc, toUtc
3. fetchRows(admin, fromUtc, toUtc)
 - SELECT registros_ponto WHERE horario_acao BETWEEN fromUtc AND toUtc
 - SELECT profiles (todos)
 - SELECT setores (todos)
 - JOIN: r.user_id → profiles → setores → r.setor = setores.nome
4. fetchRecipientEmails(admin, setorParam)
 - SELECT user_roles WHERE role = 'admin' → adminIds
 - SELECT setores → filtrar: nome IN ('GESTOR', 'GESTOR - CD', 'GESTOR - LOJA')
 - CD: aceita setores com "GESTOR" E SEM "LOJA"
 - LOJA: aceita setores com "GESTOR" E SEM "CD"
 - Para cada admin: verificar se setor_id pertence a gestorIds
5. Para cada SETOR (CD e/ou LOJA):
 - Filtrar rows por setor (r.setor.toUpperCase().includes(match))
 - Se 0 rows → skip
 - Download fotos (limit 40) + avatares
 - reconstructRondas() → agrupar por user_id e ciclo
 - buildPdf() → gerar PDF com pdf-lib
 - Anexar ao array de attachments
6. Se attachments = 0 → retornar "Nenhum registro"
7. buildEmailHtml() → HTML do email
8. sendResend() → enviar email com anexos PDF
 - Se falhar → sendGasFallback() → GAS URL
9. Retornar JSON com status

Cron Schedule

```
CD: 0 11 * * * (11:00 UTC = 07:00 Manaus)
LOJA: 5 11 * * * (11:05 UTC = 07:05 Manaus)
```

5.3 send-monthly-report

URL: **POST** <https://rdbbayprbfqbjhfcasp.supabase.co/functions/v1/send-monthly-report>

Parâmetros

PARÂMETRO	TIPO	OBRIGATÓRIO	DESCRIÇÃO
setor	string	Não	"CD" ou "LOJA"
periodo	string	Não	Formato "YYYY-MM/YYYY-MM"

Diferenças do Relatório Diário

- Janela de tempo: mês inteiro ao invés de 24h
- Inclui dashboard estatístico com gráficos
- PDF mais detalhado com indicadores de conformidade
- Enviado no dia 1 de cada mês às 08:00 Manaus

5.4 health

URL: **GET** <https://rdbbayprbfqbjhfcasp.supabase.co/functions/v1/health>

Resposta:

```
{ "status": "ok", "timestamp": "2026-09-02T..." }
```

Usado pelo GitHub Actions para manter o Supabase ativo (keep-alive).

5.5 CORS e Headers

```
const corsHeaders = {  
  "Access-Control-Allow-Origin": "https://controle-ronda.suporte04.workers.dev",  
  "Access-Control-Allow-Headers": "authorization, x-client-info, apikey, content-type",  
};
```

6. Frontend — Rotas e Telas

6.1 Mapa de Rotas

```
/ → Redirect (baseado no role)
/login → Tela de login
/app → Layout funcionário (bottom nav)
  /app/ → Dashboard "Bater Ponto"
  /app/perfil → Perfil do funcionário
  /app/historico → Histórico de rondas
/admin → Layout admin (sidebar)
  /admin/ → Dashboard admin
  /admin/usuarios → Gestão de usuários
  /admin/setores → Gestão de setores
  /admin/registros → Registros de ronda
  /admin/observacoes → Observações
  /admin/relatorio-ronda → Relatório de ronda
  /admin/ronda-detalle/:id/:inicio → Detalhe da ronda
```

6.2 Layout Raiz (`__root.tsx`)

- `AuthProvider` (Supabase Auth context)
- `ThemeProvider` (dark/light mode)
- `Toaster` (notificações sonner)
- `Outlet` (renderiza a rota filha)

6.3 Rotas do Funcionário (`/app/*`)

Dashboard — Bater Ponto (`app.index.tsx`)

- Exibe passo atual da ronda (check_in, meio1-8, check_out_2)
- Botão para tirar foto
- `CameraCapture` abre câmera do dispositivo
- Foto recebe overlay com nome + timestamp
- Upload para Supabase Storage (`fotos_ponto`)
- Insert em `registros_ponto`
- Progresso visual do ciclo (10 etapas)

Perfil (`app.perfil.tsx`)

- Exibe nome, email, setor
- Avatar (upload para `avatars` bucket)
- Informações da conta

Histórico (`app.historico.tsx`)

- Lista de rondas do usuário
- Filtragem por data

- Status de cada ronda (em progresso / concluída)

6.4 Rotas do Administrador (`/admin/*`)

Gestão de Usuários (`admin.usuarios.tsx`) — 993 linhas

- Tabela de todos os usuários
- Criar usuário (com geração de senha)
- Editar usuário
- Excluir usuário
- Atribuição de setor
- Bulk insert de usuários (array BULK_USERS com senhas hardcoded)

Gestão de Setores (`admin.setores.tsx`)

- CRUD de setores
- Lista com contagem de usuários por setor

Registros de Ronda (`admin.registros.tsx`) — 851 linhas

- Tabela com todos os registros
- Filtros: data, setor, usuário, tipo
- Visualização de fotos inline
- Download de fotos

Observações (`admin.observacoes.tsx`) — 256 linhas

- Adicionar observações a registros
- Filtro por setor e data

Relatório de Ronda (`admin.relatorio-ronda.tsx`) — 499 linhas

- Detalhe de rondas agrupadas por usuário e data
- Detecção de ciclo (`CICLO_RONDA.length = 10`)
- PDF inline com fotos
- Envio de email com relatório

7. Lógica de Negócio

7.1 Ciclo de Ronda

O ciclo completo de ronda possui **10 etapas**:

#	TIPO_ACAO	LABEL	DESCRIÇÃO
0	check_in	Início de Ronda	Primeira foto — início da ronda
1	meio1	Meio 1 da Ronda	Foto intermediária

2	<code>meio2</code>	Meio 2 da Ronda	Foto intermediária
3	<code>meio3</code>	Meio 3 da Ronda	Foto intermediária
4	<code>meio4</code>	Meio 4 da Ronda	Foto intermediária
5	<code>meio5</code>	Meio 5 da Ronda	Foto intermediária
6	<code>meio6</code>	Meio 6 da Ronda	Foto intermediária
7	<code>meio7</code>	Meio 7 da Ronda	Foto intermediária
8	<code>meio8</code>	Meio 8 da Ronda	Foto intermediária
9	<code>check_out_2</code>	Fim de Ronda	Última foto — fim da ronda

Definição no código (`src/lib/timezone.ts`):

```
export const CICLO_RONDA: TipoAcao[] = [
  "check_in", "meio1", "meio2", "meio3", "meio4",
  "meio5", "meio6", "meio7", "meio8", "check_out_2",
];
```

7.2 Detecção de Ciclo

A função `proximaAcao()` determina qual é o próximo passo:

```
export function proximaAcao(acoesHoje: string[]): TipoAcao | null {
  const posicao = acoesHoje.length % CICLO_RONDA.length;
  return CICLO_RONDA[posicao];
}
```

Exemplos:

<code>ACOESHOJE.LENGTH</code>	<code>POSICAO</code>	<code>PROXIMAACAO</code>
0	0	<code>check_in</code>
1	1	<code>meio1</code>
5	5	<code>meio5</code>
9	9	<code>check_out_2</code>
10	0	<code>check_in</code> (novo ciclo)

7.3 Filtro por Setor

O filtro de setor opera em duas camadas:

1. Banco de dados: `registros_ponto` → `profiles` → `setores` (JOIN)

2. Código: `r.setor.toUpperCase().includes(setor.match)`

SETOR PARAM	MATCH	RESULTADO
"CD"	"CD"	Aceita "CD - GUARDAS", "GESTOR - CD"
"LOJA"	"LOJA"	Aceita "LOJA - GUARDAS", "GESTOR - LOJA"
null	—	Todos os setores

7.4 Sistema de Roles

ROLE	PERMISSÕES
admin	Acesso total: CRUD usuários, ver todos os registros, gerenciar setores, gerar relatórios
user	Bater ponto, ver próprio perfil, ver próprio histórico

Atribuição: Automática via trigger `handle_new_user()` → role `user` por padrão. **Conta admin:** Apenas `suporte04@baeetrica.com.br` (configurado manualmente).

7.5 Timezone

Todo o sistema opera em **America/Manaus (UTC-4)**:

```
export const MANAUS_TZ = "America/Manaus";
export const MANAUS_UTC_OFFSET = "-04:00";
```

Conversões importantes:

- `toManausISO("2026-08-19")` → `"2026-08-19T00:00:00-04:00"`
- `formatManaus(date)` → `"dd/MM/yyyy HH:mm:ss"` em Manaus

8. Componentes

8.1 Componentes Customizados

CameraCapture.tsx (205 linhas)

Captura de câmera para bater ponto.

PROP	TIPO	DESCRIÇÃO
onCapture	<code>(file: File) ⇒ void</code>	Callback com o arquivo capturado
disabled	<code>boolean</code>	Desabilitar câmera

AdminSidebar.tsx

Sidebar de navegação do admin. Links para todas as rotas `/admin/*`.

EmployeeBottomNav.tsx

Navegação inferior do funcionário. Links para `/app`, `/app/perfil`, `/app/historico`.

ThemeToggle.tsx

Toggle para alternar entre modo claro e escuro.

8.2 Componentes shadcn/ui (46 componentes)

Todos os componentes estão em `src/components/ui/`:

COMPONENTE	USO NO PROJETO
accordion	FAQ, seções colapsáveis
alert	Notificações
alert-dialog	Confirmações de exclusão
aspect-ratio	Proporção de imagens
avatar	Fotos de perfil
badge	Status de rondas
breadcrumb	Navegação
button	Todos os formulários
calendar	Seleção de datas
card	Cards de dashboard
carousel	Galeria de fotos
chart	Gráficos Recharts
checkbox	Seleção múltipla
collapsible	Seções colapsáveis
command	Busca
context-menu	Menu de contexto
dialog	Modais
drawer	Drawer lateral
dropdown-menu	Menus suspensos
form	Formulários React Hook Form
hover-card	Preview de informações

input	Campos de texto
input-otp	Código OTP
label	Labels de formulário
menubar	Menu superior
navigation-menu	Navegação principal
pagination	Paginação
popover	Popovers
progress	Barras de progresso
radio-group	Seleção única
resizable	Painéis redimensionáveis
scroll-area	Áreas com scroll
select	Dropdowns
separator	Separadores
sheet	Drawer de detalhes
sidebar	Sidebar admin
skeleton	Loading states
slider	Sliders
sonner	Notificações toast
switch	Toggles
table	Tabelas de dados
tabs	Abas
textarea	Áreas de texto
toggle	Botões toggle
toggle-group	Grupos de toggle
tooltip	Dicas

9. Lib e Utilitários

ARQUIVO	FUNÇÕES EXPORTADAS	DESCRIÇÃO
<code>auth.tsx</code>	<code>AuthProvider</code> , <code>useAuth()</code>	Contexto de autenticação Supabase
<code>config.ts</code>	<code>SUPPORT_EMAIL</code>	<code>"suporte04@baelettrica.com.br"</code>

<code>config.server.ts</code>	—	Variáveis de ambiente server-side
<code>timezone.ts</code>	<code>toManausISO()</code> , <code>formatManaus()</code> , <code>formatHora()</code> , <code>formatData()</code> , <code>isSameDayManaus()</code> , <code>CICLO_RONDA</code> , <code>TIPO_ACAO_LABEL</code> , <code>TIPO_ACAO_ORDEM</code> , <code>proximaAcao()</code> , <code>acoesDoCicloAtual()</code> , <code>contarCiclosConcluidos()</code>	Utilitários de timezone e ciclo
<code>storage.ts</code>	<code>getSignedPhotoUrl()</code>	URLs assinadas para fotos
<code>photoOverlay.ts</code>	<code>overlayPhoto()</code>	Sobreposição de nome + timestamp via Canvas
<code>date-filters.ts</code>	Filtros de data predefinidos	Hoje, ontem, semana, mês
<code>report.functions.ts</code>	<code>sendReport()</code>	Chamada ao Edge Function de relatório
<code>admin-users.functions.ts</code>	<code>createUser()</code> , <code>updateUser()</code> , <code>deleteUser()</code>	CRUD de usuários
<code>access.functions.ts</code>	<code>syncUserAccess()</code>	Sincronização de acesso
<code>utils.ts</code>	<code>cn()</code>	Utility para classes Tailwind
<code>theme.tsx</code>	<code>ThemeProvider</code>	Contexto de tema dark/light

10. Segurança

10.1 Inventário de Chaves

CHAVE	VALOR (RESUMIDO)	ONDE ESTÁ	SENSÍVEL?
Supabase Anon Key	<code>eyJhbG...anon...Q_w</code>	wrangler.toml, client.ts, deploy.yml, .env	BAIXO (protegida por RLS)
Supabase Service Role Key	<code>eyJhbG...service_role...SBdas</code>	Code.gs (HARDCODED) , client.server.ts (env)	CRÍTICO
Resend API Key	<code>re_it6KZMRb_...</code>	Supabase Secrets (env)	ALTO
Cloudflare API Token	—	GitHub Secrets	ALTO
GAS Deployment URL	<code>https://script.google.com/macros/s/AKfycbw-.../exec</code>	send-daily-report/index.ts	MÉDIO

Supabase Project ID	<code>rdmbayprbfqbjhfqcasp</code>	Múltiplos arquivos	BAIXO
Supabase Access Token	<code>sbp_a3da4c3b ...</code>	Usado apenas em scripts locais	ALTO

10.2 Variáveis de Ambiente

Client-side (expostas ao browser)

VARIÁVEL	FONTE	VALOR
<code>VITE_SUPABASE_URL</code>	.env, wrangler.toml	<code>https://rdmbayprbfqbjhfqcasp.supabase.co</code>
<code>VITE_SUPABASE_PUBLISHABLE_KEY</code>	.env, wrangler.toml	Chave anon (JWT)
<code>VITE_SUPABASE_PROJECT_ID</code>	.env, deploy.yml	<code>rdmbayprbfqbjhfqcasp</code>

Server-side (não expostas)

VARIÁVEL	FONTE	USO
<code>SUPABASE_URL</code>	Workers env	URL do Supabase
<code>SUPABASE_PUBLISHABLE_KEY</code>	Workers env	Chave anon
<code>SUPABASE_SERVICE_ROLE_KEY</code>	<code>wrangler secret put</code>	Acesso admin ao banco
<code>RESEND_API_KEY</code>	Supabase Secrets	API Resend
<code>Deno.env.get("SUPABASE_URL")</code>	Auto-provided	Edge Functions
<code>Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")</code>	Auto-provided	Edge Functions

10.3 Problemas de Segurança Identificados

P0 — CRÍTICO

#	PROBLEMA	ARQUIVO	AÇÃO
1	Service Role Key hardcoded em plain text	<code>google-apps-script/Code.gs:11</code>	Mover para Script Properties. Rotacionar chave imediatamente
2	11 senhas de usuários hardcoded no client-side	<code>src/routes/admin.usuarios.tsx:137-203</code>	Senhas ficam no bundle JS enviado ao browser. Mover para server-side

P1 — ALTO

#	PROBLEMA	ARQUIVO	AÇÃO
---	----------	---------	------

3	Senha padrão <code>postgres</code> no Docker	<code>.env</code> , <code>docker-compose.yml</code>	Alterar para senha forte
4	Edge functions com <code>verify_jwt = false</code>	<code>supabase/config.toml</code>	Habilitar JWT verification

P2 — MÉDIO

#	PROBLEMA	ARQUIVO	AÇÃO
5	Chave anon hardcoded em 8+ arquivos	Múltiplos	Centralizar em <code>.env</code>
6	GAS URL com hash secreto hardcoded	<code>send-daily-report/index.ts:22</code>	Mover para <code>Deno.env</code>
7	<code>cadastro-computadores-v2</code> escuta em <code>0.0.0.0</code>	<code>server.cjs:34</code>	Bind em <code>127.0.0.1</code>

10.4 Recomendações de Remediação

- 1. Rotacionar Service Role Key:** Gerar nova chave no Supabase Dashboard → Settings → API → `service_role`
- 2. Mover senhas para server-side:** Usar server functions do TanStack para criar usuários
- 3. Habilitar JWT verify:** Alterar `verify_jwt = true` no `supabase/config.toml`
- 4. Remover hardcoded anon key:** Usar variáveis de ambiente em todos os arquivos
- 5. Adicionar `.env` ao `.gitignore`:** Já está, mas verificar que não foi commitado acidentalmente

11. Deploy e CI/CD

11.1 Cloudflare Workers (Frontend)

URL Produção: `https://controle-ronda.suporte04.workers.dev`

Configuração

```
# wrangler.toml
name = "controle-ronda"
compatibility_date = "2026-05-20"
compatibility_flags = ["nodejs_compat"]
main = "@tanstack/react-start/server-entry"
```

Deploy Manual

```
npm run build
npx wrangler deploy
```

Deploy Automático

O push para a branch `main` dispara o GitHub Actions `deploy.yml` :

1. Checkout do código
2. Setup Node.js 22
3. `npm ci`
4. `npx vite build` (com env vars do Supabase)
5. `wrangler deploy` (com Cloudflare API token)

11.2 Supabase Edge Functions

Deploy Manual

```
cd supabase/functions
supabase functions deploy send-daily-report --project-ref rdmbayprbfqbjhfqcasp
supabase functions deploy send-monthly-report --project-ref rdmbayprbfqbjhfqcasp
supabase functions deploy health --project-ref rdmbayprbfqbjhfqcasp
```

Configuração de Secrets

```
supabase secrets set RESEND_API_KEY=re_it6KZMRb... --project-ref rdmbayprbfqbjhfqcasp
```

Configuração

```
# supabase/config.toml
[functions.health]
verify_jwt = false

[functions.send-daily-report]
verify_jwt = false

[functions.send-monthly-report]
verify_jwt = false
```

11.3 Google Apps Script

Deploy

1. Abrir `https://script.google.com/home/projects`
2. Selecionar projeto existente ou criar novo
3. Colar código de `google-apps-script/Code.gs`
4. Salvar `appscript.json` no editor de manifest
5. Executar manualmente uma vez (para autorizar permissões)

6. Criar implantação → Web App → Executar como: eu → Acessar: qualquer pessoa
7. Copiar URL de implantação

Conta

- **Email:** `suporte.baeletrica@gmail.com` (pessoal Gmail)
- **NÃO é Google Workspace** — GmailApp funciona normalmente
- **Script ID:** `1SNDyuhthes3c7DY-r0z07WFAXiqH7kb-qSn2LpbikwVxzpoYFYTL6V27`

11.4 GitHub Actions

`deploy.yml` — Deploy Frontend

```
name: Deploy to Cloudflare Workers
on:
  push:
    branches: [main]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: npm }
      - run: npm ci
      - run: npx vite build
      env:
        VITE_SUPABASE_URL: ...
        VITE_SUPABASE_PUBLISHABLE_KEY: ...
      - uses: cloudflare/wrangler-action@v3
        with:
          apiToken: ${ secrets.CLOUDFLARE_API_TOKEN }
          command: deploy
```

`supabase-reports.yml` — Keep-Alive

```
name: Supabase Reports Keep-Alive
on:
  schedule:
    - cron: '0 6 * * *' # 06:00 UTC diariamente
jobs:
  health-ping:
    runs-on: ubuntu-latest
    steps:
      - run: curl -s https://rdbmayprbfqbjhfqcasp.supabase.co/functions/v1/health
```

11.5 Sequência de Deploy

Deploy de Frontend (automático)

1. Push para `main`
2. GitHub Actions inicia
3. `npm ci` → instala dependências
4. `vite build` → gera bundle
5. `wrangler deploy` → faz upload para Cloudflare Workers
6. Verificação: `curl https://controle-ronda.suporte04.workers.dev`

Deploy de Edge Function (manual)

1. Alterar `supabase/functions/*/index.ts`
2. `supabase functions deploy {nome} --project-ref rdmbayprbfqbjhqcasp`
3. Verificar: `curl -X POST https://rdmbayprbfqbjhqcasp.supabase.co/functions/v1/{nome}`

Deploy de Migração (manual)

1. Criar arquivo em `supabase/migrations/YYYYMMDDHHMMSS_nome.sql`
2. `supabase db push --project-ref rdmbayprbfqbjhqcasp`

11.6 Rollback

Frontend

```
# Cloudflare Workers não tem rollback automático
# Reverter commit e push:
git revert HEAD
git push origin main
# GitHub Actions faz deploy automático da versão anterior
```

Edge Function

```
# Não há versão anterior automaticamente
# Manter backup do index.ts anterior
# Re-deploy com código anterior:
supabase functions deploy send-daily-report --project-ref rdmbayprbfqbjhqcasp
```

Migração

```
-- Rollback manual (não automático)
-- Desfazer mudanças da migration
DROP TABLE IF EXISTS nova_tabela;
DROP TYPE IF EXISTS novo_enum;
-- etc.
```

12. Cron e Agendamento

12.1 pg_cron — Jobs Ativos

JOB ID	NOME	SCHEDULE	UTC	MANAUS	STATUS
2	ba-report-monthly	0 12 1 * *	Dia 1 às 12:00	Dia 1 às 08:00	ATIVO
10	ba-warmup-edge	59 10 * * *	10:59 diário	06:59 diário	ATIVO
11	ba-report-daily-cd	0 11 * * *	11:00 diário	07:00 diário	ATIVO
12	ba-report-daily-loja	5 11 * * *	11:05 diário	07:05 diário	ATIVO

12.2 Cronograma Diário

```
06:59 Manaus (10:59 UTC) → ba-warmup-edge → GET /health (keep-alive)
07:00 Manaus (11:00 UTC) → ba-report-daily-cd → POST send-daily-report {setor:"CD"}
07:05 Manaus (11:05 UTC) → ba-report-daily-loja → POST send-daily-report {setor:"LOJA"}
```

12.3 Cron Mensal

```
Dia 1, 08:00 Manaus (12:00 UTC) → ba-report-monthly → POST send-monthly-report
```

12.4 Diagnóstico de Falhas

Verificar status dos jobs

```
SELECT jobid, jobname, schedule, active FROM cron.job ORDER BY jobid;
```

Verificar execuções recentes

```
SELECT * FROM cron.job_run_details ORDER BY start_time DESC LIMIT 10;
```

Verificar respostas HTTP

```
SELECT id, status_code, content, created
FROM net._http_response
ORDER BY created DESC LIMIT 10;
```

Reenviar manualmente

```
# CD
curl -X POST https://rdbmayprbfqbjhqcasp.supabase.co/functions/v1/send-daily-report \
-H "Authorization: Bearer {ANON_KEY}" \
-H "Content-Type: application/json" \
-d '{"setor":"CD"}'

# LOJA
curl -X POST https://rdbmayprbfqbjhqcasp.supabase.co/functions/v1/send-daily-report \
-H "Authorization: Bearer {ANON_KEY}" \
-H "Content-Type: application/json" \
-d '{"setor":"LOJA"}'
```

13. Troubleshooting

13.1 Tabela de Problemas Comuns

#	PROBLEMA	CAUSA PROVÁVEL	SOLUÇÃO
1	Email não chega	Resend API key inválida ou expirada	Verificar <code>RESEND_API_KEY</code> via <code>supabase secrets list</code>
2	Email não chega (CD ok, LOJA falha)	Erro transiente no fetchRows (profiles/sets vazios)	Retry automático (já implementado). Verificar logs
3	PDF não gera	Foto corrompida ou timeout no download	Verificar logs. Aumentar timeout se necessário
4	Usuário não consegue bater ponto	RLS bloqueou INSERT	Verificar se <code>user_id = auth.uid()</code> no request
5	Login falha	Supabase Auth configurado incorretamente	Verificar URL e anon key no client

6	Deploy falha	Cloudflare token expirado	Renovar <code>CLOUDFLARE_API_TOKEN</code> no GitHub Secrets
7	Build erro	Dependência quebrada	<code>rm -rf node_modules && npm ci</code>
8	Cron não dispara	pg_cron job inativo	Verificar <code>cron.job</code> e re-schedule se necessário
9	0 registros no relatório	Time window não bate com dados	Verificar timezone. Usar <code>override_email</code> para teste
10	Fotos não carregam	Storage bucket privado sem política	Verificar policies do bucket <code>fotos_ponto</code>
11	Erro "Unauthorized"	JWT verification negado	Verificar <code>verify_jwt</code> no config.toml
12	Badge com linhas no PDF	Linha separadora desenhada por cima do badge	Atualizar edge function (fix já deployado)
13	GAS não envia	Workspace não permite GmailApp	Conta deve ser pessoal (não Workspace)
14	Observações não salvam	RLS bloqueia UPDATE para não-admins	Verificar role do usuário
15	Histórico vazio	Usuário não tem registros	Verificar se registros_ponto tem dados para o user_id

13.2 Comandos de Diagnóstico

Verificar Edge Functions

```
# Health check
curl https://rdbmayprbfqbjhqfqcasp.supabase.co/functions/v1/health

# Teste CD
curl -X POST https://rdbmayprbfqbjhqfqcasp.supabase.co/functions/v1/send-daily-report \
  -H "Authorization: Bearer {KEY}" \
  -H "Content-Type: application/json" \
  -d '{"setor":"CD","override_email":"seu@email.com}"'
```

Verificar Banco de Dados

```
# Contar registros
supabase db query "SELECT count(*) FROM registros_ponto" --project-ref rdbmayprbfqbjhqfqcasp

# Verificar setores
supabase db query "SELECT id, nome FROM setores" --project-ref rdbmayprbfqbjhqfqcasp

# Verificar users admin
supabase db query "SELECT p.nome, p.email FROM profiles p JOIN user_roles ur ON p.id = ur.user_id"
```

Verificar Cron

```
# Status dos jobs
supabase db query "SELECT * FROM cron.job" --project-ref rdmbayprbfqbjhfcasp

# Últimas execuções
supabase db query "SELECT * FROM cron.job_run_details ORDER BY start_time DESC LIMIT 5" --project-ref rdmbayprbfqbjhfcasp
```

14. Runbooks

14.1 Deploy de Edge Function

Pré-requisitos: Supabase CLI instalado e logado

```
# 1. Navegar até o diretório
cd supabase/functions

# 2. Deploy da function
supabase functions deploy send-daily-report --project-ref rdmbayprbfqbjhfcasp

# 3. Verificar deploy
curl -X POST https://rdmbayprbfqbjhfcasp.supabase.co/functions/v1/send-daily-report \
  -H "Authorization: Bearer {ANON_KEY}" \
  -H "Content-Type: application/json" \
  -d '{"setor":"CD","override_email":"seu@email.com"}'

# 4. Verificar email recebido
```

14.2 Migração de Banco

```
# 1. Criar arquivo de migration
# Nome: YYYYMMDDHHMMSS_descricao.sql
# Exemplo: 20260902120000_add_new_column.sql

# 2. Escrever SQL idempotente
# Usar IF NOT EXISTS / IF EXISTS para segurança

# 3. Push para o banco
supabase db push --project-ref rdmbayprbfqbjhfcasp

# 4. Verificar
supabase db query "SELECT * FROM information_schema.tables WHERE table_name = 'nova_tabela'"
```

14.3 Rotação de Segredos

Resend API Key

```
# 1. Gerar nova chave em https://resend.com/api-keys
# 2. Atualizar no Supabase
supabase secrets set RESEND_API_KEY=nova_chave --project-ref rdmbayprbfqbjhfqcasp
# 3. Testar
curl -X POST https://rdmbayprbfqbjhfqcasp.supabase.co/functions/v1/send-daily-report \
  -H "Authorization: Bearer {KEY}" \
  -H "Content-Type: application/json" \
  -d '{"setor": "CD", "override_email": "seu@email.com"}'
```

Supabase Service Role Key

```
# 1. Gerar nova chave no Supabase Dashboard → Settings → API
# 2. Atualizar no Cloudflare Workers
echo "nova_chave" | npx wrangler secret put SUPABASE_SERVICE_ROLE_KEY
# 3. Atualizar no GAS (Code.gs) – MELHOR: usar Script Properties
# 4. Testar todas as functions
```

Cloudflare API Token

```
# 1. Gerar novo token em https://dash.cloudflare.com/profile/api-tokens
# 2. Atualizar no GitHub Secrets
# Settings → Secrets → Actions → CLOUDFLARE_API_TOKEN → Edit
```

14.4 Envio Manual de Relatório

```
# Enviar para email específico (teste)
curl -X POST https://rdmbayprbfqbjhfqcasp.supabase.co/functions/v1/send-daily-report \
  -H "Authorization: Bearer {SERVICE_ROLE_KEY}" \
  -H "Content-Type: application/json" \
  -d '{
    "setor": "CD",
    "override_email": "seu@email.com"
  }'

# Enviar para todos os destinatários (produção)
curl -X POST https://rdmbayprbfqbjhfqcasp.supabase.co/functions/v1/send-daily-report \
  -H "Authorization: Bearer {SERVICE_ROLE_KEY}" \
  -H "Content-Type: application/json" \
  -d '{"setor": "LOJA"}'
```

15. Apêndices

15.1 Glossário

TERMO	DEFINIÇÃO
BA Elétrica	Empresa proprietária do sistema
CD	Centro de Distribuição
LOJA	Unidade de varejo
Ronda	Sequência de 10 registros fotográficos
Ciclo	Ronda completa (check_in → check_out_2)
Passo	Etapa individual da ronda
Bater Ponto	Registrar um passo com foto
Gestor	Admin que recebe relatórios
Edge Function	Função server-side no Supabase (Deno)
pg_cron	Agendamento PostgreSQL
Resend	Serviço de email transacional
GAS	Google Apps Script
RLS	Row-Level Security
JWT	JSON Web Token
CRUD	Create, Read, Update, Delete
SSR	Server-Side Rendering
SPA	Single Page Application

15.2 Contatos e Responsáveis

FUNÇÃO	CONTATO
Suporte Técnico	suporte04@baeletrica.com.br
Desenvolvimento	Equipe de Desenvolvimento BA Elétrica
GitHub	https://github.com/suporte04-BA/controle-ronda
Produção	https://controle-ronda.suporte04.workers.dev
Supabase Dashboard	https://supabase.com/dashboard/project/rdbmayprbfqbjhfqcasp

15.3 Referências

- [TanStack Start Docs](#)
 - [Supabase Docs](#)
 - [Cloudflare Workers Docs](#)
 - [Resend API Docs](#)
 - [pdf-lib Docs](#)
 - [shadcn/ui Docs](#)
 - [Tailwind CSS Docs](#)
 - [Google Apps Script Docs](#)
-



BA Elétrica — Sistema de Controle de Ronda

Documento gerado em 02/09/2026 — Versão 1.0.0

CONFIDENCIAL — Uso interno